

DAYTONA HACKSPRINT TOKYO

TikTok for Work

An AI-native communication OS
for you, your AI agents, and your team.

Approve it, and it's already done.

● Daytona · every agent has its own computer

● Neo4j · the decision graph

WHY NOW

Slack and Notion were built for humans typing to humans.

Work now happens between people *and their AI agents*.

An agent can't sit in a channel. A decision can't be swiped.

Nothing that gets approved starts moving on its own.

We should work AI-native: you talk only to your own AI, the AIs work across the organization, and what reaches you is only what you need to decide.

An approval costs a small team twice

3

tools opened before you can say yes.

Will it break anything? What did we decide last time? Whose work does it collide with? The answers live in GitHub, Slack and last month's notes, never on the request.

2 days

lost after you say yes.

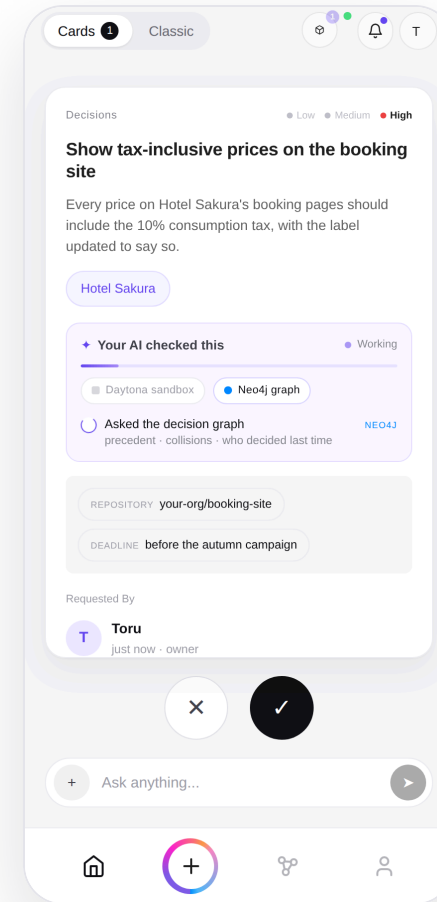
Approval is where the work starts. Someone still writes the change, runs the tests, opens the PR. "Approved, not started" is the state most decisions sit in.

Teams of 3–10 people running several businesses and repositories feel both, every day.

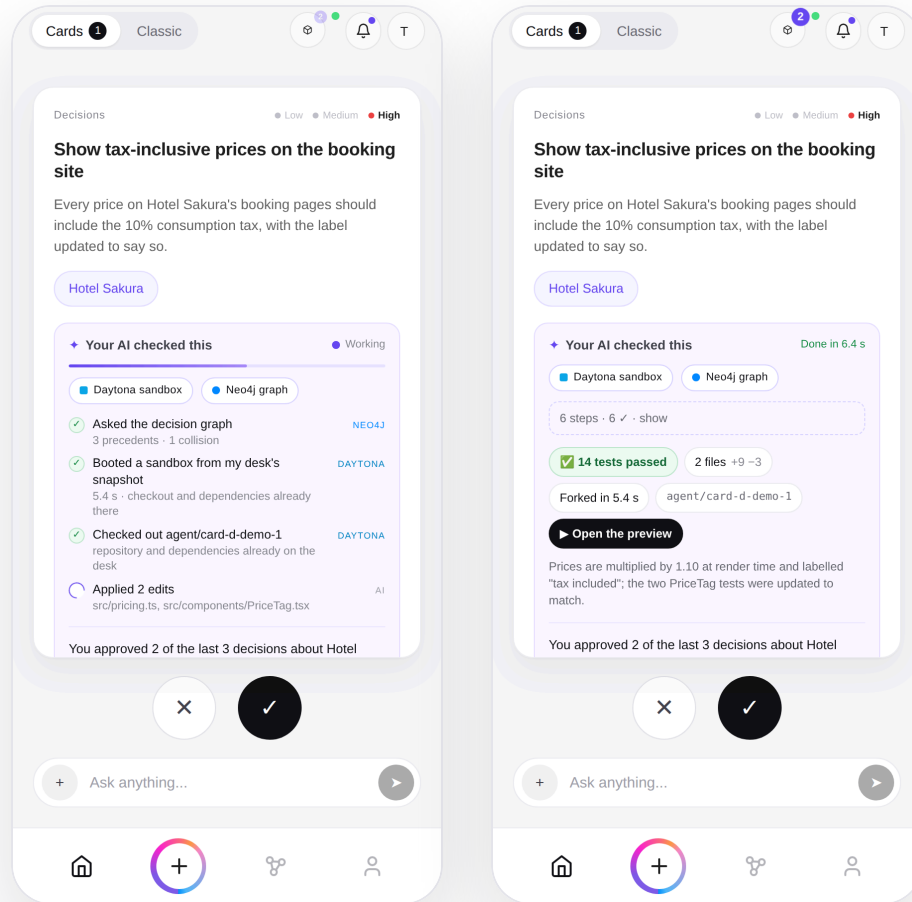
Decisions arrive like a feed. Clear them with a swipe.

- You talk only to **your own AI**. No channels, no DMs, no inbox.
- It routes the request across the org and hands it to the right person as a **Decision Card**.
- One card per screen, most urgent first. **Swipe right to approve, left to decline.**
- Every outcome lands in GitHub and is filed under the business it is about.

Open the feed, and the decision you need to make is already there.



The card carries evidence. The approval executes itself.



Before you open the card, your AI has already:

- **Tried it** — booted a sandbox from its own desk, made the change, ran every test, put up a live preview.
- **Remembered** — asked the decision graph what was decided before and whose open work this collides with.

Swipe right, and the tested branch is pushed and the pull request opens.

Real run: 10 seconds from card to evidence. 12 tests. Preview served from the sandbox.

Card → evidence → swipe → pull request



- Toru asks for tax-inclusive prices. The card lands with Tanaka.
- Tanaka's AI asks the graph, boots a sandbox from its desk, edits, tests, previews. **~10 s.**
- Tanaka swipes right. The branch is pushed, PR #12 opens, Toru hears back.
- The next request already knows about this one.

Try it yourself, no sign-in:

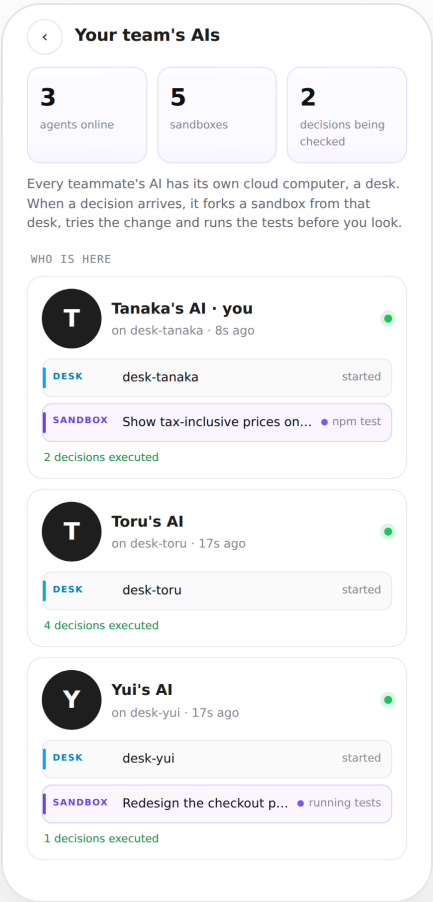
torutesu.github.io/Tiktokforwork/?demo

Video:

torutesu.github.io/Tiktokforwork/demo.webm

Every agent has its own computer

DAYTONA	IN THE PRODUCT	ON SCREEN
Persistent sandbox per agent, <code>desk-<user></code>	The agent's own machine. The runner itself is deployed into it.	Fleet: DESK · started
<code>createSnapshot</code> / <code>fork</code>	One sandbox per decision, booted from the desk with checkout and dependencies in place.	"Booted a sandbox from my desk's snapshot · 2.5 s"
<code>executeCommand</code> , <code>fs.uploadFile</code>	Apply the model's edits, run the tests, read the diff.	"Applied 1 edit · 12 passed"
<code>getSignedPreviewUrl</code>	The changed app, served from the sandbox, one tap away.	► Open the preview
Labels, auto-stop, auto-delete	Approve pushes from the same sandbox, then deletes it. Abandoned ones expire.	Fleet counts, graph Sandbox nodes
Another persistent sandbox	Neo4j itself runs on Daytona behind its preview link.	<code>graph-neo4j</code>



Decisions become context for the next decision



```
(:Person) - [:OWNS] -> (:Agent) - [:RUNS_ON] -> (:Desk)
(:Sandbox) - [:FORKED_FROM] -> (:Desk)
(:Sandbox) - [:CHECKED] -> (:Decision)
(:Person) - [:REQUESTED] -> (:Decision) - [:ASSIGNED_TO] -> (:Person)
(:Decision) - [:DECIDED_BY {action, at}] -> (:Person)
(:Decision) - [:ABOUT] -> (:Business)
(:Decision) - [:TOUCHES] -> (:Repo) - [:PRODUCED] -> (:PR)
```

// per card, in milliseconds

precedent: same business, decided, newest first

collision: same repo, still open, and who holds it

- The agent seeds the graph from the org's cards and writes every decision and PR back as it happens.
- It publishes the live neighbourhood to every screen: the org, drawn the way Neo4j holds it.

HOW IT WORKS

The recipient's AI, made real

1 · REQUEST

Toru tells their AI

"Show tax-inclusive prices on the booking site."

2 · ROUTE

The relay picks the approver

Roles and approval authority send it to Tanaka.

3 · EVIDENCE

Tanaka's AI does the homework

NEO4J precedent, collisions

DAYTONA sandbox, edit, test, preview

4 · DECIDE

Tanaka swipes right

One look at the evidence.

5 · EXECUTE

Branch pushed, PR opened

From the same sandbox.
Graph updated. Toru gets a card back.

The relay lets only a card's recipient update it. The agent joins **as the recipient**, over the same AG-UI protocol the feed clients speak, so it can enrich that person's cards and nobody else's. Relay and clients unchanged; the agent lives on its Daytona desk.

TRY IT

It's live.

torutesu.github.io/Tiktokforwork/?demo

scripted walkthrough, no sign-in

torutesu.github.io/Tiktokforwork

the real feed, sign in with an email code

github.com/Torutesu/Tiktokforwork

source: web, agent, relay, demo site

Working today: feed · routing · real-time relay · GitHub sync · evidence panel · decision graph · fleet screen · agent with desk, snapshot sandboxes, edits, tests, preview, push, PR · Neo4j seeding, queries and live graph.

Next: blast radius of an approval · several sandboxes trying alternatives in parallel · dry-runs for non-code decisions · graph-informed routing.